

Trabajo Práctico 2

TEMA

- Herencia.

OBJETIVOS

Que el estudiante adquiera las siguientes competencias específicas:

- Crear proyectos reutilizando clases mediante la aplicación de relaciones de herencia y representando mediante diagramación UML de clases.
- Usar la estructura de datos dinámica lista con POO utilizando el lenguaje de programación Java, aplicando herencia.
- Usar, en forma eficiente, la diagramación UML en la documentación de aplicaciones de software.

RECURSOS BIBLIOGRÁFICOS

Libro	
GOODRICH – TAMASSIA. Estructuras de Datos y Algoritmos en Java. Editorial Continental. Mexico. 2002	
JOYANES AGUILAR - ZAHONERO MARTINEZ. Programación en JAVA 2 - Algoritmos, Estructuras de Datos y Programación orientada a objetos. Mc Graw Hill -2002.	
DEITEL P.J. JAVA- como programar"- Pearson- Prentice Hall. Septima edición 2008	
JOYANES AGUILAR LUIS ZAHONERO MARTINEZ IGNACIO. "Estructuras de Datos en JAVA". Mc Graw Hill.2008.-	
WEISS MARK ALLEN. Estructuras de datos en JAVA. Pearson- 2013	
Software	Plataforma Java y última versión del IDE NetBeans

CONSIGNAS

- Identifique clases, atributos, métodos y objetos para resolver el problema aplicando POO. Confeccione una tabla con lo identificado.
- Aplique diagramación de clases UML.
- Expresar el algoritmo solución en diagramas de flujo, para cada uno de los métodos identificados.
- Desarrolle la codificación en Java de las clases, utilizando el entorno de desarrollo (IDE) Netbeans.
- Envío de la codificación mediante plataforma CUV.
- La resolución de la identificación de componentes del problema, la diagramación de flujo y la diagramación UML, deben estar completadas en la carpeta del alumno.

Ejercicio 1

Tomando como referencia el ejercicio 4 del Trabajo Práctico N.º 1, se solicita reorganizar la solución existente incorporando una estructura basada en herencia que permita reutilizar información común.

Se deberá definir una clase Persona como superclase de las clases Cliente y Empleado, identificando los atributos y comportamientos compartidos y separándolos de aquellos específicos de cada tipo.

Asimismo, se deberá almacenar los objetos creados utilizando una lista dinámica implementada manualmente.

Aplicar las siguientes consideraciones:

- clase Persona con atributos comunes (por ejemplo: DNI, nombre, apellido);
- clases Cliente y Empleado que hereden de Persona;
- uso de constructores que invoquen al constructor de la superclase;
- redefinición de toString() para ser convocado por un método mostrar() que mostrará la información completa;
- implementación de una lista dinámica para almacenar objetos;
- una clase principal que permita:
 - agregar personas (clientes o empleados),
 - recorrer la lista y mostrar los datos;
- diagramación UML de clases, incluyendo la relación de herencia y la estructura de la lista.

Trabajo Práctico 2

Ejercicio 2

Se desea modelar distintos dispositivos tecnológicos: Celular, Computadora y Tablet, los cuales comparten características generales.

Para evitar duplicación de código, se deberá definir una clase base Dispositivo, de la cual derivarán las demás clases.

Los dispositivos deberán almacenarse en una lista dinámica propia, permitiendo su gestión dentro del sistema.

Aplicar las siguientes consideraciones:

- clase Dispositivo con atributos comunes (marca, modelo, estado);
- métodos encender() y apagar() definidos en la superclase;
- subclases Celular, Computadora y Tablet con atributos propios;
- uso de herencia para reutilizar código;
- implementación de una lista dinámica de dispositivos;
- operaciones básicas:
 - agregar dispositivos,
 - recorrer y mostrar información;
- clase principal de prueba;
- diagrama UML de clases que represente la jerarquía y la lista.

Ejercicio 3

Se requiere modelar un sistema de empleados, diferenciando entre EmpleadoAsalariado y EmpleadoPorHora, ambos derivados de una clase base Empleado.

Cada tipo de empleado presenta diferencias en la forma de determinar su salario, pero comparten información general.

Los empleados deberán almacenarse en una lista dinámica implementada en el proyecto.

Aplicar las siguientes consideraciones:

- clase Empleado con atributos comunes;
- subclases EmpleadoAsalariado y EmpleadoPorHora;
- atributos específicos en cada subclase;
- métodos que permitan obtener el salario;
- uso de constructores con llamada a la superclase: la clase hija inicializa primeramente los atributos heredados mediante la invocación al constructor de la clase padre usando super(), y luego inicializa sus propios atributos;
- implementación de una lista dinámica de empleados;
- recorrido de la lista para mostrar información;
- diagramación UML completa.

Ejercicio 4

Se desea representar un conjunto de animales pertenecientes a distintas especies: Perro, Gato y Pájaro.

Todos los animales comparten características generales, pero cada tipo presenta particularidades propias. Se deberá modelar una clase base Animal y las subclases correspondientes.

Los animales deberán almacenarse en una lista dinámica, permitiendo su gestión.

Aplicar las siguientes consideraciones:

- clase Animal con atributos comunes;
- subclases Perro, Gato y Pajaro;
- métodos definidos en la superclase (por ejemplo: hacerSonido(), moverse());
- redefinición de métodos en subclases;
- implementación de una lista dinámica de animales;
- recorrido de la lista mostrando información;

Trabajo Práctico 2

- clase principal de prueba;
- diagrama UML que refleje herencia y estructura de datos.

Ejercicio 5

Se desea implementar un sistema que permita representar distintos tipos de pagos: PagoTarjeta y PagoEfectivo.

Todos los pagos comparten información general (como el monto), pero presentan diferencias en su forma de ejecución. Se deberá definir una clase base Pago y sus subclases. Simular el proceso de realizar un pago de un monto determinado.

Los pagos deberán almacenarse en una lista dinámica.

Aplicar las siguientes consideraciones:

- clase Pago con atributos comunes;
- subclases PagoTarjeta y PagoEfectivo;
- métodos que simulen la realización del pago;
- uso de herencia para reutilizar código;
- implementación de una lista dinámica de pagos;
- operaciones:
 - agregar pagos,
 - recorrer y mostrar datos;
- clase principal;
- diagrama UML.

Ejercicio 6

La empresa Mercedes Benz desea gestionar información sobre distintos tipos de vehículos: Automóvil, Camión, Colectivo y MotoVehículo.

Se deberá modelar una clase base Vehiculo y sus subclases correspondientes, organizando la información mediante herencia.

El sistema deberá incluir un menú de opciones y utilizar una lista dinámica para almacenar los vehículos.

- Aplicar las siguientes consideraciones:
- clase Vehiculo con atributos comunes;
- subclases Automovil, Camion, Colectivo y MotoVehiculo;
- atributos específicos en cada subclase;
- implementación de una lista dinámica de vehículos;
- crear un menú de opciones que permita agregar nueva información sobre un vehículo, eliminar la información de un vehículo y mostrar por pantalla la información completa de un vehículo determinado.
- recorrido de la lista para mostrar información;
- uso de toString() para visualización, convocado por método mostrar();
- diagramación UML de clases.

PRODUCTO ESPERADO

- Entrega grupal mediante la plataforma del CUV de la resolución en proyecto Java de los ejercicios 5 y 6.
- Desarrollo de la diagramación UML de clases y diagramación de flujo de todos los ejercicios. Individual. Carpeta del alumno.